

The Builder — how AgentForge plans, writes, verifies, repairs and edits the application

The ``agents/`` subsystem that turns an approved brief into a running Next.js 16 + MongoDB application — it writes the plan, streams every source file onto disk, proves the result compiles and serves, repairs what it broke, and afterwards edits the app from a click, a drawn annotation or an uploaded picture.

Repository	https://github.com/Ravindu200232/full-stack-ai-web-developer
Branch	builder-agent
Scope	agents/
Written	2026-08-14

Contents

Contents	2
1. Overview	3
2. What it does	3
3. Technology and tools	9
4. How it works	11
1. 1 — Plan	11
2. 2 — Scaffold	11
3. 3 — Write every file	11
4. 4 — Draw the pictures	11
5. 5 — Static repair	11
6. 6 — Analyzer, round one	11
7. 7 — Install, reset, compile	12
8. 8 — Serve, test and repair at runtime	12
9. 9 — Analyzer, runtime round	12
10. 10 — Editing: resolve	12
11. 11 — Editing: write	12
12. 12 — Editing: prove it still works	12
5. The code, file by file	13
6. Why it is built this way	15
7. Interfaces	18

1. Overview

The Builder is eighteen Python modules totalling 14,861 lines under ``agents/``, driven by the editing and orchestration routes in ``server.py``. Its centre is ``ArchitectAgent`` (4,581 lines in ``agents/architect.py``), and its module docstring states the design plainly: nothing is templated. The raw user prompt goes straight to the model, the model writes a big ``plan.md``, and then it generates every application file itself by calling a ``write_file`` tool. Only configuration is Python-generated — ``package.json``, ``next.config.mjs``, ``jsconfig.json``, ``tailwind.config.js``, ``postcss.config.js``, ``app/globals.css``, ``lib/mongodb.js``, ``env.local`` and, when the plan says there are accounts, the whole Better Auth trio. Those paths sit in ``NEXT_PROTECTED`` and a write to any of them is refused out loud.

Generation is one uninterrupted conversation rather than a turn per phase. ``build_app()`` states the plan once, hands over the whole file list grouped under its tasks, and then says "Continue" every time the model stops. What is still outstanding is computed from the disk (``_outstanding()``, ``_current_task()``), never from a counter — so a model that jumps ahead is simply never asked for a file it already wrote, and a machine that lost power mid-build resumes from the next unwritten file with the same thread reloaded from ``.agentforge/convo.json``. The transport is a streaming text protocol, ``<write_file path="app/tasks/page.jsx"> ... </write_file>``, parsed incrementally by ``FileStreamParser`` so tokens reach the studio as they arrive and an unterminated file is still salvaged on ``close()``. Native Ollama tool calls are accepted too, but the text protocol is primary because it works with every model.

The rest of the subsystem exists because "the model finished" and "the app works" are different claims. ``agents/exports.py`` parses every named import against the target module's real export surface — the check that catches the bug ``next build`` cannot see, where a 500 at request time looks like "login reaches the dashboard and bounces back". It also shells out to the project's own esbuild to prove every file parses. ``agents/analyzer.py`` reads the finished project back and compares it against the plan, splitting the work deliberately: file paths in Python (zero false positives, no tokens), meaning by the model, and credentials by a real HTTP POST at the running app. ``agents/bugfixer.py`` arbitrates before it repairs — a generated unit test is the least authoritative gate in the pipeline, so the model must emit ``VERDICT :: test|code :: <evidence>`` before any write, and the verdict decides the write allowlist. ``agents/security.py`` asks the one question nothing else asks: is this safe to put in front of people.

After the build, the same machinery serves the editing tools. Click an element and ``agents/picker.py`` resolves which file renders it — the route's import closure first, literal text and class matching second, the model only when the leading candidate is not clear. Draw over a region and ``agents/capture.py`` drives Playwright to screenshot exactly that region with the strokes painted into the page, because no browser can rasterise an iframe from the parent document. Point at a picture and ``agents/images.py`` redraws it through a local Fooocus, or ``/image-swap`` puts your own file there instead. Every one of those writes is preceded by a snapshot on disk and followed by ``verify_after_edit``, so an undo point always exists and a broken edit is caught by the app itself rather than by guessing at the size of the diff.

2. What it does

Feature	What it does	Where
The planning pass, plan.json, and the role-coverage replan	<code>`make_plan()`</code> streams a markdown plan (Overview, Data Model, Routes, Server/Client Split, Packages, Demo Accounts, Component Tree, Images, Design System, Build Tasks, Definition of Done) followed by one fenced JSON block. <code>`_extract_plan_json`</code> takes the last parseable block; <code>`_normalise_plan`</code> coerces it into <code>`phases`</code> with per-file <code>`kind`</code> (server/client/route), <code>`done_when`</code>, <code>`covers`</code>, <code>`images`</code>, <code>`demo_accounts`</code> and <code>`dependencies`</code>, inserts <code>`app/page.jsx`</code> if the plan forgot it, and drops any <code>`FR-`</code> id the planner invented that is not in the brief (a real run cited FR-035/036/037 against a 33-requirement SRS). <code>`_roles_without_areas`</code> then counts signed-in route prefixes structurally against the roles in <code>`demo_accounts`</code> and triggers one replan when three or more roles share fewer than <code>`roles - 1`</code> areas. Prose goes to <code>`plan.md`</code>, the parsed object to <code>`_agentforge/plan.json`</code>.	agents/architect.py – NEXT_PLANNER_SYSTEM, make_plan, _extract_plan_json, _normalise_plan, _roles_without_areas, _save_plan_json
The streaming write protocol	<code>`FileStreamParser`</code> splits the token stream into prose and file bodies on <code>`<write_file path="...">` / `</write_file>`</code>, emitting <code>`on_file_start`</code> / <code>`on_file_token`</code> / <code>`on_file_end`</code> as it goes. <code>`_safe_flush_len`</code> holds back only as many trailing characters as could be a split tag, so text streams to the UI without leaking a partial tag. <code>`close()`</code> salvages a file the model never terminated, and records it in <code>`_last_truncated`</code> so the next turn is told to write it again in full before carrying on.	agents/architect.py – FileStreamParser, _safe_flush_len, _strip_fence, _fix_doubled_tags, _run_write_loop
One conversation, driven by what is on disk	<code>`build_app()`</code> opens a single thread, states the plan and task list once, then loops <code>`_run_write_loop`</code> with a plain "Continue" plus what is still missing, grouped under its task with that task's goal and <code>`done_when`</code>. Up to <code>`max(6, min(40, files + 6))`</code> turns; two consecutive turns with no file written hands what exists to the repair passes. Phase events for the UI timeline and the QA agent are re-derived from the files that have landed (<code>`_sync_phases`</code>), not from the turn that is running, and <code>`final=True`</code> closes out a half-finished phase using the files that do exist so it still gets tests for its real half.	agents/architect.py – build_app, _run_write_loop, _outstanding, _current_task, _sync_phases, _outstanding_by_task, _continue_prompt
write_file: refusals, extension canonicalisation, duplicate chrome	One choke point for every write. <code>`canonical_path`</code> forces <code>`js`</code> for <code>`lib/**`</code> and <code>`app/**/route.*`</code> and <code>`jsx`</code> for components and page/layout/error/not-found, and deletes the twin extension — the bundler resolves <code>`jsx`</code> first, so both existing makes one dead code. It refuses a shadowing <code>`app/api/auth/**`</code> handler, a "use client" file importing <code>`@/lib/auth`</code>, a fetch of the non-existent <code>`/api/auth/me`</code>, <code>`middleware.js`</code> alongside <code>`proxy.js`</code>, a page/route clash in one segment, and anything in <code>`NEXT_PROTECTED`</code>. It restores a dropped <code>`import './globals.css`</code>, and <code>`_drop_layout_duplicates`</code> removes a component the root layout already renders when it stands alone on its own line in a page. Refusals are recorded by <code>`_refuse`</code>, replayed as <code>`_refusal_note`</code> next turn, and after 3 retries of one path (or 10 total) <code>`_stuck_on_refusals`</code> raises <code>`_RefusalLoop`</code> to end the turn.	agents/architect.py – write_file, canonical_path, _refuse, _refusal_note, _stuck_on_refusals, _drop_layout_duplicates, _drop_tests_for, NEXT_PROTECTED
Context compaction, thread persistence and resume	<code>`_trim_convo`</code> compacts oldest-first inside a 62% budget: strip source snapshots, then replace written file bodies with <code>`// [written earlier — N lines, still on disk]`</code>, then drop whole turns. <code>`save_convo`</code> persists the compacted thread atomically to <code>`_agentforge/convo.json`</code> (keeping one pre-strip backup at <code>`_agentforge/convo.pre-strip.json`</code>), and <code>`resume()`</code> reloads plan and thread and continues without re-planning. <code>`_write_atomic`</code> uses a temp file and <code>`os.replace`</code> because a build saves its thread after every turn, so a machine that loses power is overwhelmingly likely to do it mid-write.	agents/architect.py – _trim_convo, _stub_files, _strip_snapshot, save_convo, load_convo, _back_up_thread_once, _write_atomic, resume, unfinished

Feature	What it does	Where
Deterministic Next.js post-passes	<code>`apply_next_fixes()`</code> runs six mechanical repairs in dependency order: rewrite Pages-Router habits (<code>`next/router`</code> → <code>`next/navigation`</code> , <code>`<Link to=`</code> → <code>`href=`</code> , strip <code>`next/head`</code> , strip absolute localhost URLs); delete <code>`onError`/`onLoad`</code> from Server Component images; prepend <code>`use client`</code> where hooks, handlers or framer-motion appear (skipped when the file exports <code>`metadata`</code> , or already has a misplaced directive); add <code>`export const dynamic = 'force-dynamic'`</code> to any page or route importing <code>`@/lib/mongodb`</code> ; restore <code>`app/layout`</code> if it lost <code>`<html>`/`<body>`/`globals.css`</code> ; and heal <code>`lib/auth.js`</code> 's <code>`trustedOrigins`</code> on every build and update, because Python does not reload an imported module and a server started before the rule existed would go on scaffolding the old file.	<code>agents/architect.py</code> – <code>apply_next_fixes</code> , <code>fix_next_imports</code> , <code>strip_server_img_handlers</code> , <code>enforce_use_client</code> , <code>enforce_dynamic</code> , <code>verify_root_layout</code> , <code>verify_auth_config</code>
Static lint and one repair turn	<code>`lint_generated()`</code> collects misplaced <code>`use client`</code> , TypeScript syntax, <code>`react-router-dom`</code> , hand-built <code>`MongoClient`</code> , <code>`next/head`</code> , a default export in a <code>`route.js`</code> , <code>`pages/`</code> files, esbuild syntax failures, uninstalled packages, client/server mixes, undefined JSX identifiers, orphaned components, event handlers in server files, lucide icons passed as props, broken named imports, and planned-but-unwritten files. <code>`repair_lint()`</code> hands them to the model in exactly one turn — deliberately not a loop, because lint-fix-relint is how a build starts thrashing — followed by <code>`_fix_boundary_props()`</code> if any server component still passes a handler across the boundary.	<code>agents/architect.py</code> – <code>lint_generated</code> , <code>repair_lint</code> , <code>_fix_boundary_props</code> , <code>client_server_mix</code> , <code>undefined_jsx_components</code> , <code>orphaned_components</code> , <code>event_handlers_in_server</code> , <code>component_props_to_client</code> , <code>missing_planned_files</code>
Export and syntax checking	<code>`check_named_imports`</code> parses every local import (<code>`./`</code> , <code>`../`</code> , <code>`@/`</code>) and compares the named bindings against the target's real export surface, following <code>`export * from`</code> transitively; <code>`next/server`</code> , <code>`next/headers`</code> , <code>`next/navigation`</code> and <code>`next/cache`</code> have hard-coded surfaces. Biased hard against false positives: exports read from both raw and comment-stripped source and unioned, imports read from the stripped source only, and a module re-exporting from something unresolvable suppressed entirely. <code>`check_syntax`</code> writes a throwaway Node script that requires the project's own esbuild and calls <code>`transformSync(src, {loader:'jsx'})`</code> on every generated file, returning <code>`(problems, reason)`</code> so "no problems" and "not checked" stay distinguishable.	<code>agents/exports.py</code> – <code>check_named_imports</code> , <code>effective_exports</code> , <code>parse_exports</code> , <code>parse_imports</code> , <code>resolve_local</code> , <code>group_messages</code> , <code>check_syntax</code> , <code>syntax_messages</code> , <code>FRAMEWORK_EXPORTS</code>
Import repair: repoint first, generate second	<code>`redirect_dead_imports()`</code> moves an import only when the path resolves to nothing AND exactly one file in the project exports every name being imported — deterministic and provable, never a guess. <code>`repair_missing_imports()`</code> then generates whatever local module is still unresolved, in one model turn, matching the named/default exports the importing files ask for.	<code>agents/architect.py</code> – <code>redirect_dead_imports</code> , <code>repair_missing_imports</code> , <code>_named_exporters</code> , <code>_resolves</code>
The command tool	The model may emit <code>`<run_command>npm install date-fns</run_command>`</code> . <code>`agents/commands.py`</code> parses with <code>`shlex.split`</code> and executes an argv list with no shell, so <code>`&&`</code> , <code>` `</code> , <code>`>`</code> and backticks are arguments that fail the allowlist rather than operators. Allowlisted programs (npm/npx/node/yarn/pnpm) with per-program subcommands, banned flags (<code>`-g`</code> , <code>`--prefix`</code> , ...), plain-package-name validation, project-dir cwd, timeout, output truncation and a 12-call cap. Refusals are fed back to the model deliberately — silently dropping them invites a retry loop. <code>`_already_satisfied`</code> skips an install that would be a no-op, each costing about a minute.	<code>agents/commands.py</code> – <code>validate</code> , <code>CommandRunner</code> ; <code>agents/architect.py</code> – <code>run_requested_commands</code> , <code>_already_satisfied</code> , <code>install_planned_deps</code> , <code>install_unresolved</code> , <code>unresolved_packages</code> , <code>sync_dependencies</code>

Feature	What it does	Where
Version-matched documentation, two ways	From Next 16.2 the `next` package ships its own docs at `node_modules/next/dist/docs/` — 444 files, 3.9 MB on 16.3.0. Too big to paste (~1.1 M tokens), so `docsindex` puts a ~15-line hand-picked topic table in the system prompt and serves whole pages on ` <read_docs topic="use-client"></read_docs> >`, three per turn, 24 KB each. Separately, `nextdocs` notices `nextjs.org/docs/messages/<slug>` links in build or server output, fetches that markdown once, caches it in memory and under `~/.agentforge/docs/messages/`, and appends it to the fix prompt — the model cannot fetch it itself because `commands.py` bans curl, wget and PowerShell.	agents/docsindex.py — TOPICS, index_block, read, serve; agents/nextdocs.py — slugs_in, fetch, guidance_for
The analyzer's deterministic scan	`scan()` compares the finished project against its plan and emits `Finding(severity, code, message, path, fix)` for: missing planned files (and scaffold placeholders still in place), broken imports, `await searchParams` self-destructuring, session cookie mismatches, unique indexes in the seed, prop-contract breaks, exposed credentials, seed volume, dead endpoints, dead links, stray directives, uninstalled packages, seed races, count-guarded seeds, `redirect('/login')` for the wrong role, seeding behind an auth guard, auth origin, `user._id` on a Better Auth session, auth completeness, chrome in the root layout, and password-hash leaks — then folds in `arch.lint_generated()` as `major`.	agents/analyzer.py — scan, Finding, AnalyzerReport, enumerate_routes, and the individual check methods
Runtime verification: probe every route, then log in for real	`probe_routes` GETs every non-dynamic page and every GET API route — plain HTTP is sub-second, so nothing is sampled (the tester's five-page alphabetical sample once skipped `/login`, `/orders`, `/products`, `/search` and `/signup`). 401/403 from an API route is correct, not a fault; only 5xx and 404 are. `verify_credentials` then POSTs the plan's demo credentials at the login endpoint found by looking for `bcrypt.compare` — the only way to catch a wrong password hash, since a bad hash returns a perfectly valid 401. A failure rewrites the seed and, when `allow_reseed` is set, drops the database so the corrected seed can run, then rescans and re-verifies.	agents/analyzer.py — run_runtime, probe_routes, verify_credentials, find_login_endpoint, demo_credentials, _credentials_from_seed, _announce_credentials; agents/mongo.py — reset_project_db
The unbuilt-promises audit	Asks the model for behaviour the plan promised in prose that no file implements — the gap `planned_paths()` leaves open. Measured on a real build: the plan promised "a sales dashboard with date-range filters driven by the URL", `app/admin/page.js` was written, served 200, contained zero `searchParams`, and the scan reported no problems. The anti-hallucination guard is the feature: every finding must quote the promise verbatim from `plan.md` (normalised substring, minimum 25 characters) and name a file that exists, both checked in Python, so the worst case is silence rather than a repair pass rewriting working code.	agents/analyzer.py — unbuilt_promises, PROMISE_RE, _norm, orphan_components, diagnose
Allowlisted analyzer repair	`repair()` builds a write allowlist from the deterministic missing-file list plus the paths of blocker findings and their `extra` lists, and throws away any write outside it — except a genuinely new file under `app/`, `components/` or `lib/`, which is allowed and logged. The prompt carries the plan, a one-line-per-file inventory, `app/layout.js` plus every `lib/` helper, the current contents of files being edited, Next's own error-page guidance, and — when the caller has one — the dev server's stack frames, which turn "HTTP 500" into a file and a line. `run()` scans, repairs and rescans, stopping as soon as a round stops reducing the blocker count.	agents/analyzer.py — repair, run, inventory, route_table, _read_for_model, _budget_chars

Feature	What it does	Where
The bug fixer's verdict protocol	Before any write the model must emit `VERDICT :: test code unclear :: <evidence>`; `test` may write only the test file, `code` only the target, anything else is discarded, and because the stream is parsed incrementally a model that writes first and explains afterwards gets the safe default. Deterministic priors are shown as observations rather than imposed — except an unparseable test (quarantined, no model call) and a file nobody may edit. Weakening is checked in Python after the write: `it.skip`, `expect(true).toBe(true)`, assertions halved, or passing cases dropped, each rejected with a reason that is remembered and fed to the next attempt. Tiers escalate only when repeating the current strategy has stopped paying — tier 1 adds the refusal reason and more neighbourhood, tier 2 grants permission to fix the component instead of the test.	agents/bugfixer.py – SYSTEM, fix, prior, _commit, _weakened, _lost_cases, _refuse, editable, _threw_in, missing_export, imports_target
Runtime repair, planned then written	`_repair_runtime` splits the job: `FeaturesAgent.plan_repair` reads the whole inventory and names the files a failure spans, then `BugFixerAgent.fix_runtime` writes exactly those with that plan as the allowlist. It used to be one `arch.update(all_errors)`, with the model choosing files and rewriting them in the same breath while seeing only an 18-file snapshot — on a forty-file project the file a stack frame names is regularly not in that snapshot. `_commit_runtime` drops the file-count budget and changed-line ceiling and keeps only the two guards a rewrite never justifies: truncation, and a dropped export other files import.	server.py – _repair_runtime, run_build_fix_loop, terminal_faults; agents/bugfixer.py – fix_runtime, _commit_runtime, RUNTIME_SYSTEM
Section / pencil edit with a real screenshot	The user draws over the running preview. `capture_region` launches headless Chromium at the iframe's own CSS viewport (so tablet and mobile modes reproduce exactly), POSTs the demo credentials into the context first because most drawing happens on an authenticated screen, navigates, restores the scroll position, paints the strokes into the page as a red SVG polyline overlay via `page.evaluate`, and screenshots the padded bounding box — re-capturing at a reduced device scale factor if the base64 exceeds 1.5 MB. `_vision_model` borrows a vision-capable model for that single call when the build model has none; with no vision at all the drawing is still used to locate the region.	agents/capture.py – capture_region, strokes_bounds, _OVERLAY_JS, PENCIL_SYSTEM, CaptureResult; server.py – run_pencil_edit, _pencil_write_round, _vision_model
Pictures: generated by the plan, by the markup, by prompt, or uploaded	`ImageAgent` discovers a running Fooocus, reads its `/config` to rebuild the 153-argument generate payload, and overrides exactly four values — prompt, aspect ratio, image count, seed — driving either the Gradio 3 WebSocket queue or the Gradio 4 SSE queue, with `GENERATE_TIMEOUT = 600` as the ceiling. `run_image_stage` draws the plan's `images`; `_fill_missing_images` then scans the source for `/generated/*.png` references the plan never listed — including `\${p.slug}` template forms resolved against seeded values — and draws them from their alt text. `/image-edit` redraws the picture a user points at with `force=True` and rewrites every file holding the old `src`; `/image-swap` does the identical rewrite with the user's own file, needing no image model, no language model and no network.	agents/images.py – ImageAgent, _load_template, _predict, _predict_ws, ASPECTS, slug; server.py – run_image_stage, _fill_missing_images, run_image_edit, run_image_swap, image_agent, _find_foocus_config, PICTURES_RULE

Feature	What it does	Where
Snapshot, undo and verify-after-edit	Every picker write copies the affected files to <code>`logs/undo/<project>/<timestamp>/'</code> before the rewrite — outside the project because the project is walked to build the export zip — and emits an <code>`undo_point`</code> the UI can call <code>`/undo`</code> with. Afterwards <code>`verify_after_edit`</code> parses every file with esbuild, resolves every named import, optionally rebuilds, restarts the dev server and probes the routes, repairing each stage through the analyzer. <code>`_autofix_from_terminal`</code> then requests the edited route so Next compiles it, reads the dev server's own terminal, rewrites the one edited file up to twice, and escalates to the bug fixer — which can reach files the edit never named. <code>`qa_agent/snapshot.py`</code> does the same job in bytes for the QA rounds, recording <code>`None`</code> for a path that did not exist so a created file can be undone by deleting it.	<code>server.py</code> — <code>_snapshot</code>, <code>restore_snapshot</code>, <code>verify_after_edit</code>, <code>_autofix_from_terminal</code>, <code>terminal_faults</code>; <code>qa_agent/snapshot.py</code> — <code>FileSnapshot</code>
The security pass	The only gate that can fail an app which compiles, renders and answers every route. Checks written so a false positive is impossible: a mutating handler (<code>`PUT`/`PATCH`/`DELETE`</code> always, <code>`POST`</code> when it goes on to call <code>`updateOne`/`deleteOne`/`bulkWrite`/...`</code>) with no session signal; a page under a role-gated section that checks nothing itself; a <code>`NEXT_PUBLIC_*SECRET PASSWORD TOKEN`</code> variable; a password compared or stored with no hashing; a request body handed straight to Mongo as a filter, or an interpolated <code>`\$where`</code>; a non-literal <code>`dangerouslySetInnerHTML`</code>. Plus <code>`npm audit --json`</code>, which answers the question nobody can answer by reading code. <code>`GET /api/products`</code> with no session check is deliberately NOT reported; <code>`POST`</code> to the same path is.	<code>agents/security.py</code> — <code>SecurityAgent.run</code>, <code>unguarded_routes</code>, <code>unguarded_pages</code>, <code>exposed_secrets</code>, <code>plain_passwords</code>, <code>query_injection</code>, <code>unsafe_html</code>, <code>audit</code>

3. Technology and tools

Technology	Version	What it does here	Why
Ollama	—	The model transport. <code>`ArchitectAgent._stream`</code> iterates <code>`client.chat_stream(model, messages, tools=..., options={'temperature','top_p','num_ctx'}, keep_alive='10m', think=...)`</code> , accumulating <code>`prompt_eval_count`/`eval_count`</code> and surfacing a heartbeat every 30 seconds that distinguishes characters written from thinking tokens from empty chunks.	One client serves local and cloud models; <code>`is_cloud_model`</code> and <code>`max_context`</code> decide how aggressively the conversation is compacted, and a <code>`model`</code> override on one call is what lets the pencil tool borrow a vision-capable model for the single request that carries an image.
Next.js	16.3.0, pinned exactly	The generated app's framework — App Router, JavaScript only, run on webpack via <code>`next dev --webpack`</code> / <code>`next build --webpack`</code> .	Moved up from 15.5 so the model can read the framework's own documentation: 16.2+ ships the full docs inside the package. Webpack rather than the new default Turbopack because <code>`Failed to compile`</code> / <code>`Module not found`</code> / <code>`Can't resolve`</code> are the strings the build fix loop parses, and all three were verified present with <code>`--webpack`</code> on 16.3.
React	19.0.8	The component runtime of the generated app; the server/client boundary is what most of the house rules are about.	Pinned alongside Next 16.3. <code>`reactStrictMode: false`</code> in the generated <code>`next.config.mjs`</code> , because <code>StrictMode</code> double-invokes effects and that double-inserts seed rows.
MongoDB Node driver	6.21.0	The generated app's only database access, reached exclusively through the scaffolded <code>`lib/mongodb.js`</code> (<code>`getDb`</code> , <code>`getCollection`</code> , <code>`serialize`</code> , <code>`ObjectId`</code>).	mongodb 7 requires Node ≥ 20.19 and the vendored Node is 20.18.1; 16.3 itself needs only $\geq 20.9.0$, so 6 avoids a Node bump.
Tailwind CSS	3.4.19	The only styling mechanism the generated app may use — no styled-components, no extra CSS imports.	Tailwind 3 rather than v4 because v4's CSS-first config is not what models emit.
Better Auth	1.6.26, with @better-auth/mongo-adapter 1.6.26	Scaffolded whole when the plan has accounts: <code>`lib/auth.js`</code> , <code>`lib/auth-client.js`</code> and <code>`app/api/auth/[...all]/route.js`</code> . The model never writes a session cookie, never hashes a password, never makes a sessions collection.	Hand-rolled sessions are why the generated apps used to break — the cookie set under one name and read under another, <code>`user.id`</code> from a document that only has <code>`_id`</code> , a string looked up against an <code>ObjectId</code> ; all three return 200 from the login route and then drop the user back on the login form. The adapter is given <code>`client.db(...)`</code> and not <code>`client`</code> , because passing the client enables transactions, which need a replica set, and AgentForge's mongod is standalone.
esbuild	—	The syntax gate. <code>`check_syntax`</code> requires the project's own <code>`node_modules/esbuild`</code> from a throwaway Node script and calls <code>`transformSync`</code> with <code>`loader: 'jsx'`</code> on every generated file.	<code>`jsx`</code> for <code>`js`</code> too, because Next lets a <code>`js`</code> file contain JSX and the generated apps use both extensions for components. No new Python dependency — the project already has esbuild via Next.
Playwright	—	Server-side screenshot for the pencil/section tool: headless Chromium at the iframe's own CSS viewport, credentials POSTed into the context, strokes painted in by <code>`page.evaluate`</code> , clipped screenshot.	No browser can rasterise an iframe from the parent document — there is no <code>`drawImage(iframe)`</code> — and <code>html2canvas</code> would mean a CDN dependency inside an app whose whole premise is running locally, producing an approximation rather than the page. Playwright is already a dependency of <code>`agents/tester.py`</code> .

Technology	Version	What it does here	Why
Fooocus (Gradio)	—	Local image generation over HTTP — <code>`/config`</code> for the argument template, then the Gradio 3 WebSocket queue (<code>`send_hash` → <code>`send_data`</code> → <code>`process_completed`</code></code>) or the Gradio 4 SSE <code>`/queue/join` + <code>`/queue/data`</code></code> .	Fooocus exposes no REST API and no <code>`api_name`</code> , so the 153-argument generate handler is reconstructed from Fooocus's own UI description — which survives a Fooocus upgrade as long as the config still describes the UI. <code>`/run/predict`</code> alone returns a clean 200 and no picture, because generate is a generator behind the queue and the first yield is a progress bar. <code>`image_host`</code> is configurable because the workstation with the GPU is not always the one running AgentForge.
mongod (vendored)	fallback 8.3.7	The database the generated apps use, resolved or downloaded to <code>`~/agentforge/mongo`</code> and run on 127.0.0.1:27017; an already-listening mongod is adopted, never killed.	AgentForge's promise is that nothing has to be installed. Only <code>`mongod`</code> is extracted, not the archive: the Windows ZIP is 923 MB of which 91% is debug symbols, and because fastdl serves <code>`Accept-Ranges: bytes`</code> the central directory is read over HTTP ranges and the single 32.5 MB member inflated — 37.7 MB transferred instead of 922.6 MB. Readiness is the port plus the "Waiting for connections" log marker, so no pymongo and no motor.
Node + npm	—	Runs the generated app, the esbuild syntax script and the database-reset script; <code>`npm`</code> installs whatever the model asks for through the command tool. Binaries resolved from <code>`LOCODE_NPM`/`NPM`</code> or PATH.	<code>`_find_npm_cmd`</code> exists because <code>`BuilderAgent._install_deps`</code> called a function that never existed, so every <code>`pipeline.py`</code> run died with NameError at npm install while the studio path never hit it — <code>`UIBuilder._install_deps`</code> overrides the method entirely.
Python stdlib — difflib, re, shlex, urllib, dataclasses	—	Every deterministic check. <code>`difflib.SequenceMatcher`</code> measures how much a rewrite changed and <code>`get_close_matches`</code> (cutoff 0.92) suggests a near-miss export; <code>`shlex.split`</code> is what makes the command tool shell-free; <code>`urllib.request`</code> does the route probes and the login POST.	The checks that matter must cost no tokens and add no dependency — file-path checking was "measured 0 false positives across all nine projects, so this needs no model". The 0.92 cutoff is tight enough to reject <code>`setSessionCookie`</code> → <code>`createSession`</code> , a rename that would compile and then fail at runtime with the wrong behaviour.
lucide-react, framer-motion	—	The two UI libraries the generated app is told to use, scaffolded into <code>`package.json`</code> by default and named repeatedly in the house rules and in the checks.	Icons are the single most common boundary failure — a lucide icon passed as a prop from a Server Component throws "Only plain objects can be passed to Client Components" once per card while the page still returns 200, so a four-stat dashboard throws four times and ships green.
vitest / the QA manifest	—	Read, not run, by the Builder: <code>`agentforge/qa/manifest.json`</code> maps each test file to its target, and <code>`_drop_tests_for`</code> deletes a component's tests when a write leaves that component exporting nothing.	Removing an unused component is a legitimate repair; leaving nine tests pointed at it is not — they cannot pass, no repair can make them honest, they burn every round of the QA loop, and the files are set aside at the end as a silent loss of cases. Measured on one build: exactly nine.

4. How it works

1. 1 — Plan

``ArchitectAgent.run()`` fires ``on_progress('Planning...', 5)`` and calls ``make_plan()``. The planner system prompt plus the stack rules go out, the reply streams into the studio as ``plan.md``, and the trailing JSON block becomes ``self.plan`` via ``_extract_plan_json`` → ``_normalise_plan``. ``app/page.jsx`` is inserted if the plan forgot it, invented ``FR-`` ids are dropped, images are de-duplicated and capped at 12, and the parsed plan is written to ``.agentforge/plan.json``. If three or more roles share too few signed-in areas, one replan is requested. ``start_conversation()`` then opens the single build thread with the idea and the plan.

2. 2 — Scaffold

``scaffold()`` sets ``_scaffolding = True`` (which suspends the protected-path refusals) and ``_scaffold_next()`` writes the pinned ``package.json``, ``next.config.mjs`` (StrictMode off, ``logging.browserToTerminal: 'warn'`` so client crashes arrive with a file and line), ``jsconfig.json`` with the ``@/*`` alias, Tailwind and PostCSS config, ``app/globals.css``, a root layout with ``suppressHydrationWarning`` on ``<html>`` and ``<body>``, a "Building..." placeholder home page, ``lib/mongodb.js``, ``app/api/health/route.js``, ``.env.local`` with a fresh ``BETTER_AUTH_SECRET``, ``.gitignore``, and — only when ``_needs_auth()`` says the plan has accounts — the Better Auth trio. ``write_agent_files()`` then upserts ``AGENTS.md`` around Next's own BEGIN/END marker block and points ``CLAUDE.md`` at it. ``install_planned_deps()`` installs any plan dependency the pinned allowlist does not know.

3. 3 — Write every file

``build_app()`` sends the task list, then loops. Each ``_run_write_loop`` turn appends the user message plus any refusal note from the previous turn, trims the conversation to budget, streams the reply through ``FileStreamParser``, and routes each ``</write_file>`` straight to ``write_file`` — which canonicalises the extension, removes the shadowed twin, applies the refusal guards, restores a dropped ``globals.css`` import, drops layout duplicates, writes to disk and fires ``on_file_written``. After the turn, ``<read_docs>`` tags are answered from the installed Next docs, ``<run_command>`` blocks are validated and executed with their real output fed back, and any native ``write_file`` tool call is applied. ``_sync_phases`` reports phase progress off the files that landed, which is what makes the QA agent author tests for them.

4. 4 — Draw the pictures

``run_image_stage`` generates the plan's ``images`` into ``public/generated/<key>.png``, then ``_fill_missing_images`` scans the written source for ``/generated/*.png`` references the plan never listed and draws those from their alt text. Deliberately after the files and before the build check: the markup already settles the paths, and a missing image is not a compile error, so nothing downstream waits on the GPU.

5. 5 — Static repair

``repair_missing_imports()`` (repoint, then generate), ``apply_next_fixes()`` (the six mechanical passes), ``sync_dependencies()`` (add imported packages from the pinned allowlist to ``package.json``), ``install_unresolved()`` (one model turn to ``npm install`` anything outside the allowlist), and ``repair_lint()`` — one turn on everything ``lint_generated()`` found, followed by ``_fix_boundary_props()``. ``_verify_output()`` then checks that an entry page exists and that more than two files beyond the scaffold were written.

6. 6 — Analyzer, round one

``AnalyzerAgent.run()`` scans the project against the plan, installs anything still unresolved, runs ``unbuilt_promises()`` when there are no blockers, then loops repair-and-rescan at most twice, breaking as soon as a round stops reducing the blocker count. Its writes are allowlisted to the missing/blocker paths plus genuinely new files; anything else is refused and logged.

7. 7 — Install, reset, compile

``ensure_node_deps`` runs ``npm install``; if the database is up, ``MONGO.reset_project_db`` drops the ``agentforge_*`` database left by a previous app so the seed starts clean. ``run_build_fix_loop`` then runs ``npm run build`` up to three rounds, feeding the model the filtered error text plus Next's own error pages for the slugs it names, and letting it choose between ``run_command`` for a missing package and a whole-file rewrite for anything else. Building before the dev server starts matters twice: ``next build`` and ``next dev`` share ``.next/``, and a compile error is far cheaper and far more precise than waiting for Playwright to notice a blank page.

8. 8 — Serve, test and repair at runtime

The dev server starts, ``TesterAgent`` drives the browser, and ``terminal_faults`` deduplicates the dev server's own complaints out of the log since a mark — a serialisation error on a five-card dashboard prints five near-identical blocks. Errors go to ``_repair_runtime``, which has ``FeaturesAgent.plan_repair`` name the files and ``BugFixerAgent.fix_runtime`` write exactly those, falling back to a whole-thread ``arch.update`` when the two disagree. The loop stops early when an attempt changes none of the error signatures, because further attempts would repeat it.

9. 9 — Analyzer, runtime round

``run_runtime()`` probes every page and every GET API route, POSTs the demo credentials at the login endpoint, and — on a ``BAD_CREDENTIALS`` verdict — rewrites the seed and drops the database so the corrected seed can actually run, then rescans and re-verifies. ``_announce_credentials`` reports the working accounts to the developer, never into the app.

10. 10 — Editing: resolve

An edit arrives on ``/element-edit``, ``/pencil-edit``, ``/image-edit`` or ``/image-swap``. ``_open_for_edit`` reopens the project, infers the stack from its config files, and ``load_existing()`` repopulates ``arch.files`` from disk and restores the build conversation. ``ElementResolver.resolve()`` picks the file: route closure first, literal text and attribute scoring second, the model only on a close call. ``_scope_verdict`` stops and asks when that file renders on more than one route — unless the instruction already said "everywhere" or "on this page only".

11. 11 — Editing: write

The element path sends the whole file plus the components it renders and rejects an empty or truncated reply, retrying once with the reason. The pencil path captures the annotated region, borrows a vision model if needed, and passes ``designing=True`` to ``guard_scope`` — which then refuses only an empty response or a rewrite short enough to be a truncated stream, because measuring a redesign against the original and refusing when it differs too much means refusing the thing that was asked for.

12. 12 — Editing: prove it still works

The old bytes are snapshotted to ``logs/undo/<project>/<stamp>`` and an ``undo_point`` is emitted before the file is written. ``_fill_missing_images`` draws any picture the new markup asks for. ``verify_after_edit`` parses every file, resolves every named import, restarts the dev server and (for the heavier paths) probes the routes, repairing each stage through the analyzer. ``_autofix_from_terminal`` requests the edited route, reads the dev server's terminal, rewrites the one edited file up to twice, and then hands the failure to the bug fixer — which can reach files the edit never touched.

5. The code, file by file

Path	What lives there
agents/architect.py	4,581 lines. The whole generation engine: the planner and builder system prompts, <code>FileStreamParser</code> , the plan parser and normaliser, the Next.js and Vite scaffolds, <code>write_file</code> with all its refusals, <code>build_app</code> / <code>_run_write_loop</code> , the deterministic Next post-passes, <code>lint_generated</code> , the import repairs, <code>resume</code> , <code>update</code> , and the plan/conversation persistence under <code>.agentforge/</code> .
agents/analyzer.py	2,148 lines. Reads the finished project back and compares it against the plan. <code>scan()</code> and roughly twenty-five deterministic checks, <code>enumerate_routes</code> , <code>probe_routes</code> , <code>verify_credentials</code> , the model-driven <code>diagnose</code> and <code>unbuilt_promises</code> , and the allowlisted <code>repair</code> .
agents/bugfixer.py	1,059 lines. Arbitration before repair. The verdict protocol, the deterministic priors, <code>editable()</code> , the weakening and lost-case checks, the escalation tiers, and <code>fix_runtime</code> for failures that have no test to be wrong.
agents/picker.py	Resolves the clicked element to the file that renders it (<code>ElementResolver</code> — route closure, scoring, a model tie-break) and enforces "change just that" afterwards (<code>guard_scope</code> , <code>lost_content</code>). Also <code>ELEMENT_EDIT_SYSTEM</code> and the instruction classifiers — removal, addition, retext, global, page-only — which read Sinhala and romanised Sinhala as well as English.
agents/exports.py	The named-import checker and the esbuild syntax gate. Pure Python, no dependency, shared by the architect's lint, the analyzer, the bug fixer and <code>verify_after_edit</code> .
agents/capture.py	Playwright screenshot of the region the user drew over — viewport matching, pre-login, the SVG stroke overlay injected by <code>page.evaluate</code> , the clipped capture and its 1.5 MB base64 downscale — plus <code>PENCIL_SYSTEM</code> .
agents/images.py	<code>ImageAgent</code> . Discovers a running Fooocus, reads its <code>/config</code> to build the 153-argument payload, overrides prompt/aspect/count/seed, and drives either the Gradio 3 WebSocket queue or the Gradio 4 SSE queue. Every method degrades to "no image" rather than raising.
agents/security.py	<code>SecurityAgent</code> . Unguarded write handlers, unguarded pages under a gated section, <code>NEXT_PUBLIC_</code> secrets, unhashed passwords, query injection, unsafe HTML, and <code>npm audit</code> . Emits <code>analyzer.Finding</code> so the existing repair path consumes it.
agents/commands.py	The agent's command tool: <code>validate()</code> (no shell, allowlisted programs and subcommands, banned flags, plain package names) and <code>CommandRunner</code> (project cwd, timeout, output truncation, 12-call cap).
agents/docsindex.py	The hand-picked topic table for the Next docs bundled inside the installed package, plus <code><read_docs topic="..."></code> serving. Each topic is the page that answers a rule this project already enforces and has watched models break.
agents/nextdocs.py	Notifies <code>nextjs.org/docs/messages/<slug></code> in build or server output, fetches the markdown once, caches it in memory and on disk, and returns a prompt section. Deliberately quiet — every error path returns "" so a build never fails because a docs site was unreachable.
agents/mongo.py	<code>MongoManager</code> . Resolves or range-downloads <code>mongod</code> , starts and adopts it, and <code>reset_project_db</code> — the fenced database drop that lets a corrected seed actually run (refused when the user configured their own URI, and refused for any database not named <code>agentforge_*</code> , checked in Python and again inside the Node script).
agents/features.py	<code>FeaturesAgent</code> . Adds one feature to an existing project, and supplies <code>plan_repair</code> — the file-naming half of runtime repair, which reads the whole inventory rather than the architect's fixed snapshot.

Path	What lives there
agents/refiner.py	The legacy Vite path's first agent: phrase-list, score-weighted site-type detection (<code>SITE_TYPES</code> , <code>KEYWORD_WEIGHTS</code>), an LLM refine pass hinted with that type, and the section/strategy maps. Used by <code>pipeline.py</code> and <code>run_pipeline</code> .
agents/builder.py	The legacy <code>BuilderAgent</code> — the templated Vite + React path that stitches per-component LLM output into a Python-generated <code>App.jsx</code> shell, with its own JSX sanitiser and <code>npm run build</code> fix pass. Kept because projects generated before the Next.js migration must stay editable.
agents/tester.py	The Playwright browser pass over the running app, and <code>overlay_error</code> , which reads Next's own error overlay out of the page.
agents/ollama_client.py	The streaming chat client, plus <code>is_cloud_model</code> , <code>max_context</code> and settings loading — what decides the conversation budget and whether QA is available at all.
agents/nextmcp.py	Optional client for Next's MCP endpoint — <code>call()</code> , <code>available()</code> and <code>errors()</code> , which formats recent frames for a prompt.
server.py – the editing routes	<code>run_element_edit</code> , <code>run_pencil_edit</code> , <code>run_image_edit</code> , <code>run_image_swap</code> , <code>run_page_update</code> , <code>run_feature</code> and <code>run_agent_update</code> , plus their HTTP dispatch (around lines 7285–7360) and the websocket <code>element_edit</code> / <code>pencil_edit</code> / <code>image_edit</code> messages.
server.py – the guard rails	<code>verify_after_edit</code> (syntax → imports → build → probe, each with its own repair), <code>_snapshot</code> / <code>restore_snapshot</code> , <code>_scope_verdict</code> , <code>_autofix_from_terminal</code> , <code>_element_write_round</code> , <code>_pencil_write_round</code> , <code>_edit_rules</code> , <code>PICTURES_RULE</code> , <code>_neighbours</code> , <code>_project_map</code> , <code>_layout_chain</code> , <code>_reach_label</code> .
server.py – the build orchestration	<code>run_agent_pipeline</code> , <code>run_build_fix_loop</code> , <code>_repair_runtime</code> , <code>run_image_stage</code> , <code>_fill_missing_images</code> , <code>terminal_faults</code> , <code>_open_for_edit</code> , <code>image_agent</code> , <code>_find_foocus_config</code> , <code>UIBuilder</code> , <code>run_pipeline</code> .
qa_agent/snapshot.py	<code>FileSnapshot</code> — bytes, not text, and <code>None</code> recorded for a path that did not exist, so a restore is byte-identical and a created file can be undone by deleting it. Repeated captures keep the first value, so round 2 restores to the state before round 1.
pipeline.py	The headless legacy path: <code>RefinerAgent</code> → <code>BuilderAgent</code> → <code>TesterAgent</code> , with no studio and no websocket.

6. Why it is built this way

Design decisions with their reasons. Where the source records the failure that forced a choice, that failure is given rather than a general principle.

Decision	Why
One uninterrupted conversation for the whole build, with the plan stated once and a plain "Continue" thereafter — but the work always shown grouped under its task.	Two failures pulling opposite ways. Re-prompting per phase with the plan restated invited the model to treat each phase as a fresh start, re-deciding a component's props and restyling a page it had already styled. But a bare "Continue" with fourteen files still listed makes the model budget one turn's output across all fourteen and they all come out thin. The task grouping is presentation only — the loop never tells the model to stop at a boundary.
What is outstanding is computed from the disk, never from a counter.	A cursor can disagree with reality after a crash; a directory listing cannot. It also makes "continue task 3" safe: if the model got ahead of itself and wrote task 4's files too, task 4 is simply already done and never asked for again. The same principle is why <code>resume()</code> loads the plan rather than regenerating it — a second planning pass would produce a different file list, and the half of the app already on disk would belong to a plan nobody has any more.
A refused write is recorded and replayed to the model before the next turn, and the turn is aborted once it is plainly stuck.	Every guard used to <code>return False</code> and log a line the model never sees. From inside the stream a refusal is invisible — the block was emitted, nothing came back, so the model writes it again. Measured on a real build: 3,832 refused writes, the same eleven <code>app/api/auth/*</code> paths cycling for over twelve minutes in one turn, and the build never finished.
Repoint a dead import before generating the missing module.	Generating makes every import resolve, which is right when the file was never written and badly wrong when the symbol already lives somewhere else. A repair that wrote <code>import { CartProvider } from '@components/CartContext'</code> against a project whose provider is in <code>lib/cart-context.js</code> got a second <code>CartProvider</code> generated for it, so the tree mounted one context and every <code>useCart</code> read the other. Measured — it 500'd every route. The repoint is deterministic: it moves an import only when the path resolves to nothing and exactly one file exports every name being imported.
A file's extension is decided by its role, and the twin is deleted.	The bundler resolves <code>.jsx`</code> before <code>.js`</code> , so if both <code>components/Card.js`</code> and <code>components/Card.jsx`</code> exist, <code>@/components/Card`</code> loads the <code>.jsx`</code> and the other is dead code nobody notices. <code>missing_planned_files`</code> used to manufacture exactly that collision: a plan entry <code>TaskList.jsx`</code> satisfied by a written <code>TaskList.js`</code> was reported missing, the message reached <code>repair_lint()</code> , and the model dutifully created the second stem — so the comparison is now on the extension-stripped stem.
The seed must upsert by identity and may never gate on a count.	<code>if (await usersCol.countDocuments() > 0) return`</code> reads like "seed once" and actually means "seed only while nobody has ever registered". One real signup makes the count permanently non-zero, so the demo accounts are never created while the app still advertises them, and every demo login 401s from then on. Observed exactly that way in a live database. Measured across the corpus: 13 of 13 seeded projects had it, because the builder prompt handed them the count guard as its template. <code>\$setOnInsert`</code> , not <code>\$set`</code> — a bcrypt hash differs every time it is computed, so <code>\$set`</code> would rewrite it on every cold start and could overwrite a password a real user had changed. An upsert is also atomic per document, so two simultaneous requests cannot double-insert and E11000 cannot occur.
The seed must never create a unique index.	The database survives between generations, so an index build will one day meet documents written before that field existed. Two of them have <code>sku: null`</code> , the build throws E11000, <code>ensureSeeded()</code> rejects — and because every page awaits it, every page returns 500.

Decision	Why
Demo accounts are created through Better Auth's own sign-up API, never inserted, and never rendered anywhere in the app.	Better Auth's password hashing is the only thing that produces a hash its own login will accept, so a direct <code>users.insertOne({ passwordHash })</code> gives an account that exists and can never sign in. And a login screen that lists its own passwords is not a login screen — the tool running the build shows them to the developer instead. A wrapper around <code>getSessionUser</code> is banned for the same class of reason: on one build a <code>lib/session.js</code> exporting <code>getCurrentUser</code> was imported by twelve route files and cost 32 of 46 failing tests, all reported as "expected 500 to be 401".
<code>getSessionUser()</code> returns <code>id</code> , a string, and there is no <code>user._id</code> . The check is scoped to the variable actually assigned from <code>getSessionUser()</code> .	Measured on a running generated app the session user is exactly <code>{createdAt, email, emailVerified, id, name, role, updatedAt}</code> . <code>user._id</code> is <code>undefined</code> , and <code>undefined</code> is silent: every comparison against it is false and every document written with it stores nothing. Two real failures from one build — <code>appt.vetId.toString() !== user._id</code> meant no vet could ever record a visit, and <code>ownerId: user._id</code> meant a registered pet never appeared in its owner's list. The scoping matters because <code>._id</code> on a Mongo document is correct and common: a blanket search reports 43 occurrences across the corpus of which 21 are real.
<code>lib/mongodb.js</code> connects on first use, not at module scope, and is never editable.	<code>new MongoClient(uri).connect()</code> at module scope opens a socket the moment anything imports the module — and <code>next build</code> imports every page and route during its "Collecting page data" pass. That made a running database a requirement to compile the app: it worked on a machine with mongod running and died in CI with <code>MongoServerSelectionError: connect ECONNREFUSED 127.0.0.1:27017</code> , blaming whichever route the collection worker happened to reach first. It is also the bug fixer's only <code>NEVER_CODE</code> entry, because a "fix" there is how a client/server boundary error became a webpack fallback that hid a 500.
Never <code>if (!user user.role !== 'admin')</code> redirect(<code>/login</code>), and guard every page in a section, not just its landing page.	Sharing one branch drops an allowed user with the wrong role onto the login form, which from where they are sitting is exactly what a wrong password looks like: they log in, bounce straight back, and report that login is broken. And Next inherits no guards — measured across generated apps, a clinic where <code>/reception</code> correctly bounced a nurse while <code>/reception/book</code> , the page that creates appointments, answered them 200, and a shop with eight <code>/admin/*</code> pages of which one was guarded. On the API side, twenty-five route handlers across thirty-five generated apps changed records that already existed and not one asked who was calling.
A lucide icon is never passed as a prop from a Server Component; the client component imports the icons and takes a name string. <code>onError</code> / <code>onLoad</code> on a Server Component image is stripped mechanically rather than repaired.	Passing the icon passes a function, and React answers "Only plain objects can be passed to Client Components" — once per card, so a four-stat dashboard throws four times and still returns 200. It ships green because nothing that reads status codes or the browser console ever sees it. The image handler is the same mistake wearing a defensive-programming disguise; it happened three times in one project and again in the feature added afterwards, and removing it is safe because the pictures are generated into <code>public/generated/</code> before the app runs, so there is nothing to fall back from.
The bug fixer arbitrates before it repairs, the verdict is the write allowlist, and the deterministic priors are shown to the model rather than imposed on it.	A generated unit test is the least authoritative gate in the pipeline — the only one whose own correctness is in question, having been written minutes earlier by the same model that wrote the code it judges. But priors used to outrank the model, and the cost was paid in the direction nobody sees: when the prior was wrong, a correct test was rewritten until it agreed with a bug and the suite went green by giving up what it was checking. Only two things still decide alone — a file that cannot be parsed, and a file nobody may edit — because neither is a judgement call. Whole-file rewrites of a test are the most expensive mistake available: measured, a build went 13 failures → 6 → 7 → 11 doing exactly that.

Decision	Why
An element edit is measured against its diff; a redesign is not. <code>`guard_scope(designing=True)`</code> refuses only an empty or truncated file, and the result is judged by the syntax gate, the export gate, the route probe and the bug fixer instead.	The line ceiling asked "did this change more than I expected" and threw the work away when it had — which is what a redesign always does. "The redesign changed far more than the region — nothing was written" was the outcome of asking a selection or a pencil stroke to look different, and the user got nothing at all. A guess about how many lines an edit should touch was never evidence of anything; testing the result is. The escalation from <code>`_autofix_from_terminal`</code> to the bug fixer is what made that safe, because a redesign can break a child component, a shared import or another page rendering the same file — none of which is the path the edit named.

7. Interfaces

Name	Shape
POST /element-edit	{project, prompt, element:{tag,id,className,text,attrs,chain,route,section?}, model?, think?} → {ok:true}; the work runs in a thread and reports over the websocket
POST /pencil-edit	{project, prompt, route, viewport:{w,h,mode}, scroll:{x,y}, strokes:[[{x,y},...]], element, model?, think?} → {ok:true}
POST /image-edit	{project, prompt, element:{src bg, attrs.alt}, model?, think?} → {ok:true}; redraws through Fooocus and rewrites every file holding the old src
POST /image-swap	{project, data_base64, filename, element:{src bg}} → {ok:true}. HTTP rather than the websocket because it carries a file and a multi-megabyte base64 frame is not what that socket is for
POST /undo	{project, id?} → {ok, restored:[paths], id} — omit `id` for the most recent snapshot
POST /image	{prompt, name?, project?, aspect?, seed?, force?} → {ok, file, name, data_uri, url} 409 generation off 503 no Fooocus answering 502 generation failed
POST /image-upload	The same response contract as /image with {data_base64, name filename, project?} instead of a prompt — deliberately identical so the caller cannot tell which one drew the picture. Cap UPLOAD_IMAGE_MAX = 7,500,000 bytes of base64, because Next's rewrite refuses a body over 10 MiB with a bare 500 that never reaches the process
POST /attach	{project?, filename, data_base64} → {ok, text (capped at ATTACH_TEXT_CAP = 6000 chars), ...} — text the browser folds into any editing prompt
POST /feature, /agent-update, /agent-build	{project prompt, model?, think?, qa_model?, logo?, srs_id?} → {ok:true}
WebSocket in: pencil_edit / element_edit / image_edit	The same payloads as the HTTP routes, dispatched to the same thread targets
WebSocket out: element_picked	{type, file, line, score, candidates:[{path,score}], used_model, shared_routes}
WebSocket out: undo_point	{type, id (timestamp stamp), files:[paths]} — emitted before every picker write
WebSocket out: ask	{type:'ask', kind:'scope', file, routes, route, options:[two ready-made answers]} — the scope question; the next instruction is the answer
WebSocket out: phase, file stream, progress, memory	on_phase {phase,total,title,status,goal,files,written}; on_file_start(path); on_file_token(path,tok); on_file_end(path,content); on_file_written(path,size,content); on_progress(label,pct); on_memory {turns,approx_tokens,num_ctx,cloud}
Model tool: write_file	<write_file path="app/tasks/page.jsx">...complete file...</write_file>, or the native Ollama tool call {name:'write_file', arguments:{path, content}}
Model tool: run_command / read_docs / read_file	<run_command>npm install date-fns</run_command> · <read_docs topic="use-client"/> (3 per turn, 24 KB each) · <read_file path="lib/seed.js"/> (4 per reply, budgeted against the context window)
Model reply formats	Bug fixer: `VERDICT :: test[code unclear harness :: <evidence>` before any write block. Picker tie-break: `PICK <path> <line>`. Analyzer diagnose: `FINDING <blocker major minor> <path-or-> :: <what is wrong>`. Promise audit: `PROMISE:` / `FILE:` / `MISSING:` blocks, or `NONE`. Element edit escape hatch: `NEED <path>`. Build completion: `BUILD COMPLETE`

Name	Shape
Project files the Builder owns	.agentforge/plan.json (the parsed plan — under .agentforge/ so it stays out of the file tree and the export zip) · .agentforge/convo.json (the compacted build thread, capped at 400,000 chars) · .agentforge/convo.pre-strip.json (one backup, written once) · .agentforge/qa/manifest.json (read, and pruned when a target loses its exports) · plan.md (prose, JSON block stripped) · public/generated/<key>.png
Scaffolded and refused paths	NEXT_PROTECTED = package.json, next.config.mjs, jsconfig.json, tailwind.config.js, postcss.config.js, app/globals.css, lib/mongodb.js, app/api/health/route.js, .env.local, .gitignore, AGENTS.md, CLAUDE.md, vitest.config.mjs, playwright.config.js. NEXT_SCAFFOLD additionally covers app/layout.jsx, app/page.jsx, lib/auth.js, lib/auth-client.js and app/api/auth/[...all]/route.js
Undo store	logs/undo/<project>/<YYYYmmdd-HHMMSS>/<relative path> — outside the project directory on purpose, because the project is walked to build the export zip